

Department of Computer and Software Engineering**SE: Machine Learning****Course Instructor:****Date:** May 2026**Day:****Semester:** 6th Semester**Lab 12: Model Deployment using Flask & Package Management**

Lab Title	CLO	BT	PLO	Weightage
Model Deployment using Flask REST APIs, Docker & uv				

Name	Roll Number	Report /10	Viva /5	Total /15
Hamza Anjum	BSSE23072			

Checked on: _____

Signature: _____

Objectives

- Convert an existing API from FastAPI to Flask
- Understand differences between API frameworks
- Deploy a trained model through a web interface and containerize it
- Utilize `uv` to manage modern Python environments and resolve version conflicts

Problem Statement

In the previous lab, you built a machine learning pipeline, tracked experiments using MLflow, serialized the trained model, and deployed it using a FastAPI-based REST API.

In real-world systems, different frameworks may be used depending on project requirements. Flask is a lightweight and widely used framework for building simple web applications and APIs. Furthermore, managing dependencies for complex ML packages (like MediaPipe for computer vision) often leads to version conflicts on newer Python releases.

In this lab, you will convert the FastAPI deployment into a Flask application. Following this, you will explore environment and version management using `uv` to resolve a simulated dependency conflict.

You are required to demonstrate the working system through screenshots of:

- The running Flask application and web interfaces
- Terminal outputs demonstrating dependency errors and their resolution

Part A: Understanding Existing API

Task 1: Analyze FastAPI Implementation

Review your Lab 11 FastAPI code and identify:

- **How the model is loaded:** The model was loaded globally into memory upon application startup using `joblib.load()` or `pickle.load()` from a serialized `.pkl` file.
- **How input is received:** Inputs were received via HTTP POST requests, defined and strictly validated using Pydantic schemas (`BaseModel`).
- **How predictions are returned:** The API returned predictions dynamically packaged as standard JSON responses directly from the route functions.

Part B: Flask Setup

Task 2: Create Flask Application

Create a Flask application with:

- **A home route (/):** Implemented using `@app.route('/')` to render the main HTML template for user interaction.
- **A prediction route (/predict):** Implemented using `@app.route('/predict', methods=['POST'])` to accept form data, process features, and return the inference.

Part C: Model Integration

Task 3: Load and Use Model

Load the serialized model from Lab 11 and use it to make predictions.

Ensure:

- **Correct input format:** Feature data was extracted via `request.form.values()` and cast to floats.
- **Proper handling of numerical features:** The inputs were transformed into a 2D NumPy array (`np.array(features).reshape(1, -1)`) to match the input shape expected by the scikit-learn `.predict()` pipeline.

Part D: Web Interface

Task 4: Create Input Form

Created `index.html` inside the `templates/` directory utilizing standard HTML form controls to capture feature variables. The form action was configured to submit a POST request to the `/predict` endpoint. Used Jinja2 templating syntax (`{{ prediction_text }}`) to dynamically display the prediction results back on the webpage.

Part E: Testing and Screenshots

Task 5: Demonstration

Part F: Containerization using Docker

Task 6: Create Dockerfile

A Dockerfile was created utilizing `python:3.11-slim` as the base image. The `uv` package manager was utilized for rapid dependency installation from the `requirements.txt` file to keep the image lightweight. The `EXPOSE 5000` command was mapped, and the container utilizes Gunicorn to serve the Flask app in production format.

Task 7: Build and Run Docker Image

The image was built successfully on an Apple Silicon (M1) architecture.
Screenshot Required:

Part G: AWS ECR (Elastic Container Registry)

Task 8: Create ECR Repository & Task 9: Push Docker Image

Engineering Pivot Documented: Attempted to provision the ECR repository `lab12-m1-app` using the AWS CLI. However, the institutional AWS Learner Lab IAM role (`voclabs`) enforced a billing guardrail policy (`voc-cancel-cred`), resulting in an `AccessDeniedException` that explicitly denied `ecr:CreateRepository`. To complete the registry deployment requirement, the architecture was pivoted. The local Docker daemon was authenticated with Docker Hub, the image was re-tagged (`muhammadhamzaarshad/lab12-flask-app:latest`), and successfully pushed to a public remote registry to bypass the ECR IAM blockade.

Screenshot Required:

Part H: Deployment using Elastic Beanstalk

Task 10: Launch Environment & Task 11: Verify Deployment

Engineering Constraint Documented: The deployment manifest (`Dockerrun.aws.json`) was successfully configured to map port 5000 and pull the containerized image from the remote Docker Hub registry. The Elastic Beanstalk application was initialized (`eb init`).

However, upon executing `eb create lab12-env` to provision the cloud infrastructure, an IAM Explicit Deny was triggered: `NotAuthorizedError - Operation Denied... is not authorized to perform: elasticbeanstalk:CreateApplication`.

To verify this was an institutional boundary and not a local configuration error, the deployment was tested via both the local macOS CLI and the internal AWS CloudShell. Both yielded the same IAM authorization failure, proving the pipeline was built correctly but compute allocation was locked by university budget policies.

Screenshot Required:

Part I: Modern Package Management with uv [Bonus]

Task 12: Feel the Pain — Installing with Plain pip

Experience Note: Manual `pip` installs are highly fragile because they mutate the active environment sequentially. Without a deterministic lockfile, forcing modern packages (like `pytensor` requiring newer Python versions) into legacy environments causes opaque resolution errors, missing binary wheels, or silent dependency corruption across projects.

Lab Task: Advanced Dependency Management with uv

Deliverables:

1. **Why Step 2 Failed:** `uv` strictly enforces the workspace guarantee; it threw an error because the `pyproject.toml` explicitly promised support down to Python 3.9, but `pytensor` strictly requires Python 3.10+, making the universal lockfile mathematically impossible without conditional markers.

Task 13: Resolving Version Conflicts with uv**Task 15: Running Standalone Tools with uv****Task 16: Keep Deployment Lightweight with Dev Dependencies****Task 17: Migrate requirements.txt into Modern uv Files**

Part J: Reflection

Task 18: Reflection Questions

- **What is the role of Docker in deployment?**

Docker abstracts the underlying hardware and OS, ensuring consistent execution environments. In this lab, code written and containerized on an ARM-based M1 Mac could be reliably pushed and deployed to cloud servers running standard Intel/AMD architecture, eliminating the "it works on my machine" operational paradox.

- **Why use ECR instead of storing images locally?**

Storing images locally traps the deployment artifact on a single developer's machine. ECR (or Docker Hub) provides a secure, centralized, and highly available remote registry. It natively integrates with AWS IAM for access control and allows automated deployment services (like Elastic Beanstalk or ECS) to rapidly pull the image directly into the production cloud network.

- **What are the advantages of Elastic Beanstalk?**

Elastic Beanstalk is a Platform as a Service (PaaS) that handles the heavy lifting of infrastructure provisioning. It automatically configures the EC2 compute instances, load balancers, security groups, and auto-scaling logic, allowing developers to focus purely on the application code and Docker artifact rather than manually orchestrating network infrastructure.

- **How does a tool like uv prevent "dependency hell" when working on multiple ML projects with different Python version requirements?**

`uv` prevents dependency hell by utilizing universal lockfiles and PEP 508 environment markers to conditionally install packages based on the active system architecture. Instead of bloating a global system Python, `uv` dynamically fetches required Python interpreters under the hood and builds strictly isolated, ephemeral virtual environments for each project.